Finally, you should test that admins can perform an edit and nonowners cannot. The following listing verifies this scenario.

Listing E.34 Validating admin editing and non-owner edits are rejected

```
# RiffMates/bands/tests.py
...
class TestBands(TestCase):
    ...
    def test_edit_musician(self):
        ...
        # Verify that a superuser can edit
        self.client.login(username="admin",
            password=self.PASSWORD)
        file = SimpleUploadedFile("test.gif", self.image)
        data["picture"] = file
        data["first_name"] = "Super Edited Name"
        response = self.client.post(url, data)

        self.assertEqual(302, response.status_code)
        musician = Musician.objects.last()
        self.assertEqual(data["first_name"], musician.first_name)

        # Verify that a non-owner can't edit
        self.client.login(username="member",
            password=self.PASSWORD)
        response = self.client.post(url, data)
        self.assertEqual(404, response.status_code)
```

Log in as the admin account instead.

Perform the edit actions like before but as an admin.

Validate the correct redirect response and that the edit took place.

Log in as a nonowner.

Validate that a 404 response was returned for a user without edit permissions.

Exercise 2

Write a test for the `musicians()` view that validates pagination is working. The view needs to be called multiple times with a variety of query parameters to test different settings for the number of items per page as well as the next and previous arguments. Note that the paginator's `.has_previous()` and `.has_next()` are methods, not attributes; you didn't need the parentheses in the template, but you do when calling them in code.

To verify pagination is working, you need more data, so the first part of the test is to create more musicians. I have chosen to create 9 more, making 10 in total. The following listing shows this.

Listing E.35 Creating more test data for pagination

```
# RiffMates/bands/tests.py
...
class TestBands(TestCase):
    ...
    def test_musicians(self):
        # Create 9 more Musicians (one exists from .setUp())
```